

```

1  import components.simplereader.SimpleReader;
2
3  /**
4   * use program to fulfill the pseudoscience and guess the ABCD.
5   *
6   * @author Eric Zhou
7   */
8  public final class ABCDGuesser2 {
9
10     /**
11      * No argument constructor--private to prevent instantiation.
12      */
13     private ABCDGuesser2() {
14     }
15
16     /**
17      * Repeatedly asks the user for a positive real number until the user enters
18      * one. Returns the positive real number.
19      *
20      * @param in
21      *         the input stream
22      * @param out
23      *         the output stream
24      * @return a positive real number entered by the user
25      */
26     private static double getPositiveDouble(SimpleReader in, SimpleWriter out) {
27         boolean get = false;
28         double n = -1;
29         do {
30             out.println("Please enter a positive number: "); //ask for the number
31             String s = in.nextLine();
32             if (FormatChecker.canParseDouble(s)) { //check whether it is a real number
33                 n = Double.parseDouble(s);
34                 if (n > 0) { //check whether it is positive;
35                     get = true; //change the signal
36                 } else {
37                     out.println("The number should be positive!");
38                 }
39             } else {
40                 out.println("The number should be a postive real number!");
41             }
42         } while (!get);
43         return n;
44     }
45
46     /**
47      * Repeatedly asks the user for a positive real number not equal to 1.0
48      * until the user enters one. Returns the positive real number.
49      *
50      * @param in
51      *         the input stream
52      * @param out
53      *         the output stream
54      * @return a positive real number not equal to 1.0 entered by the user
55      */
56     private static double getPositiveDoubleNotOne(SimpleReader in, SimpleWriter out) {
57         boolean get = false;
58         double n = -1;
59         do {
60             out.println("Enter a positive real number and not equal to one");
61             String s = in.nextLine();
62             if (FormatChecker.canParseDouble(s)) { //check whether it is a real number
63                 n = Double.parseDouble(s);
64                 if (n > 0) { //check whether it is positive

```

```

70         if (n == 1) { //check whether it equals to one
71             out.println("The number should not be equal to one");
72         } else {
73             get = true; //change the signal
74         }
75     } else {
76         out.println("The number should be positive and not equal to one");
77     }
78 } else {
79     out.println("The number should be positive real number");
80 }
81 } while (!get);
82 return n;
83 }
84
85 /**
86  * use the original array with the input number and get the new array.
87  *
88  * @param base
89  *         the base of the power
90  * @param exponents
91  *         the exponents of the aim
92  * @return the final new array
93  *
94  */
95 private static double[] calculate(double base, double[] exponents) {
96     double[] result = new double[exponents.length];
97     for (int i = 0; i < exponents.length; i++) {
98         result[i] = Math.pow(base, exponents[i]);
99     }
100     return result;
101 }
102
103 /**
104  * Main method.
105  *
106  * @param args
107  *         the command line arguments
108  */
109 public static void main(String[] args) {
110     SimpleReader in = new SimpleReader1L();
111     SimpleWriter out = new SimpleWriter1L();
112
113     double mu = 0; //initial the data
114     double w = 0;
115     double x = 0;
116     double y = 0;
117     double z = 0;
118
119     out.println("Please enter the constant mu you want to check: ");
120     mu = getPositiveDouble(in, out); //use method to get the data
121
122     out.println("Please enter the w: ");
123     w = getPositiveDoubleNotOne(in, out);
124
125     out.println("Please enter the x: ");
126     x = getPositiveDoubleNotOne(in, out);
127
128     out.println("Please enter the y: ");
129     y = getPositiveDoubleNotOne(in, out);
130
131     out.println("Please enter the z: ");
132     z = getPositiveDoubleNotOne(in, out);
133
134     final double[] abcd = { -5, -4, -3, -2, -1, -1.0 / 2, -1.0 / 3, -1.0 / 4, 0,

```

```

135      1.0 / 4, 1.0 / 3, 1.0 / 2, 1, 2, 3, 4, 5 };
136
137      double[] wPow = calculate(w, abcd); //use new method to get the power answer
138      double[] xPow = calculate(x, abcd);
139      double[] yPow = calculate(y, abcd);
140      double[] zPow = calculate(z, abcd);
141
142      int a = 0;
143      int b = 0;
144      int c = 0;
145      int d = 0;
146
147      final double maxx = 10000;
148      double minError = maxx;
149      double error;
150
151      double answer;
152
153      for (int i = 0; i < abcd.length; i++) { //use for loop to get the smallest error
154          for (int j = 0; j < abcd.length; j++) {
155              for (int n = 0; n < abcd.length; n++) {
156                  for (int m = 0; m < abcd.length; m++) {
157                      answer = wPow[i] * xPow[j] * yPow[n] * zPow[m];
158                      error = Math.abs(answer - mu);
159                      if (error < minError) {
160                          minError = error;
161                          a = i;
162                          b = j;
163                          c = n;
164                          d = m;
165                      }
166                  }
167              }
168          }
169      }
170
171      out.println("a = " + abcd[a]);
172      out.println("b = " + abcd[b]);
173      out.println("c = " + abcd[c]);
174      out.println("d = " + abcd[d]);
175      double finalMu = wPow[a] * xPow[b] * yPow[c] * zPow[d];
176      out.println("The best approximation is " + finalMu);
177      final int hundred = 100;
178      double finalError = Math.abs(mu - finalMu) / mu * hundred;
179
180      out.println("The relative error is " + finalError + " %");
181
182      /*
183       * Close input and output streams
184       */
185      in.close();
186      out.close();
187  }
188
189 }
190

```